

Permission Structure for Line Numbers and Calculation Logic

Change Impact Analysis - what the collector and each app viewer need so the app can show SQL / formula logic with referencing line numbers.

Showing line numbers and calculation logic requires reading the source code behind each asset: view SQL, table definitions and Tableau formulas. That is more sensitive than lineage alone. Split it across two identities:

- **A collector** (a service principal) reads the metadata broadly on a schedule and stores it.
- **The app decides per viewer** what to reveal: everyone sees the lineage, but code is shown only to people who can already read it in the source system.

1. Collector: Databricks (Unity Catalog)

Run the notebooks and the app's live mode as a dedicated service principal, for example `sp-impact-analysis`, not as a person.

Needed for	Source it reads	Grant
Tables, columns, view SQL (the line numbers in views)	<code><catalog>.information_schema.views, .columns, .tables</code>	<code>USE CATALOG</code> , <code>USE SCHEMA</code> , plus <code>BROWSE</code> (metadata only) or <code>SELECT</code>
CDE, tier and owner tags	<code>information_schema.table_tags / column_tags</code>	Same as above. <code>APPLY TAG</code> is only needed to <i>write</i> tags.
Column lineage (the derived-table hops)	<code>system.access.column_lineage</code>	<code>USE CATALOG</code> on <code>system</code> , <code>USE SCHEMA</code> on <code>system.access</code> , <code>SELECT</code> on the table
Query SQL (the line numbers in queries)	<code>system.query.history</code>	<code>USE SCHEMA</code> on <code>system.query</code> , <code>SELECT</code> on <code>history</code>
Derived-table logic	Git, dbt or DLT source (not in Unity Catalog, see caveat below)	<code>CAN READ</code> on the workspace notebooks or files, <code>CAN VIEW</code> on DLT pipelines, or a read-only Git token
Saving results	<code><catalog>.impact_analysis.*</code>	<code>USE SCHEMA</code> , <code>CREATE TABLE</code> and <code>MODIFY</code> on that schema only
Compute and secrets	SQL warehouse, <code>impact_analysis</code> secret scope	<code>CAN USE</code> on the warehouse, <code>READ</code> on the scope

```
GRANT USE CATALOG ON CATALOG insurance TO `sp-impact-analysis`;
GRANT USE SCHEMA ON SCHEMA insurance.policy TO `sp-impact-analysis`;
GRANT BROWSE ON CATALOG insurance TO `sp-impact-analysis`; -- metadata, no data
GRANT USE CATALOG ON CATALOG system TO `sp-impact-analysis`;
GRANT USE SCHEMA ON SCHEMA system.access TO `sp-impact-analysis`;
GRANT SELECT ON TABLE system.access.column_lineage TO `sp-impact-analysis`;
GRANT USE SCHEMA ON SCHEMA system.query TO `sp-impact-analysis`;
GRANT SELECT ON TABLE system.query.history TO `sp-impact-analysis`;
```

Caveats to check in your workspace:

- **BROWSE vs SELECT**: it is not certain that `BROWSE` alone makes `view_definition` visible in `information_schema.views`. Test it by querying that view as the service principal. If the definitions come back empty, grant `SELECT` on the schema, which is broader because it also allows reading data.
- **Derived-table definitions aren't in Unity Catalog**. A table built with CTAS doesn't keep its `AS SELECT` text; `SHOW CREATE TABLE` returns only the columns. The real logic lives in Git, dbt or DLT, and that's where the connector's optional `definition` field should be filled from.
- **System tables cover the whole metastore**. `system.query.history` holds every user's SQL text, which may include literal values or PII, and lineage spans all catalogs. The metastore or account admin has to grant these, and only to the service principal.

2. Collector: Tableau

Needed for	Setting
API access	A Tableau service account whose personal access token (PAT) is stored in the Databricks secret scope (already wired: <code>impact_analysis/tableau_pat</code>)
Metadata API	Enabled by default on Tableau Cloud. On Tableau Server, run <code>tsm maintenance metadata-services enable</code> .
Calculation formulas , field-to-column mappings	Account with View permission on every project in scope

The Metadata API only returns what the account itself is allowed to see. For content it can't see, fields such as `formula` come back empty or redacted. You'll get complete formulas if the account has View on every project in scope, or has the Site Administrator Explorer role. Which fields need extra licensing (Data Management) depends on your edition, so confirm with your Tableau admin.

3. Viewer: what each app user can see

The collector sees everything, so without a filter the app would show any logged-in user code they can't open in Databricks or Tableau. Use three tiers:

Tier	Shown to	Content
Lineage and governance	Everyone who can use the app	Asset names, paths, KPIs, rating, CDEs, review focus
Line locations	Everyone who can use the app	" <code>vw_policy_summary</code> L3, L10", which reveals where but not what
Code text	Only viewers who can read that object in the source system	The SQL or formula with highlighted lines. Anyone else sees: <i>"Restricted: 2 lines reference this field. Request access from <owner>"</i> .

How to check each viewer:

- **Databricks:** run a check as the signed-in user. Databricks Apps supports user authorization (running queries on behalf of the viewer), if it's enabled in your workspace. Querying `information_schema.views` as the viewer returns only the views they're allowed to see.
- **Tableau:** map the viewer's email to their Tableau user and check permissions on the workbook or data source through the REST API. A simpler option is an allow-list of groups in the governance catalog file.
- **Downloads must apply the same filter.** Otherwise the HTML, CSV or JSON export becomes a way around the redaction.
- **Stored results hold the code too.** Persisted `impact_results` include the evidence (the code), so restrict the `impact_analysis` schema to the service principal and the app.

4. What the code does today

- **Collection:** the live connector already reads `information_schema`, the tag tables, `system.access.column_lineage` and `system.query.history`, and the Tableau formulas through a PAT. The grants in sections 1 and 2 are what it needs.
- **No viewer tiers yet:** the app runs as a single identity and shows every piece of code to every user, including in the downloaded reports.
- **No derived-table definitions in live mode:** nothing fills the `definition` field from Git, dbt or DLT yet.

Suggested next step: build the viewer tiers (redacting code in the UI and in all three downloads) and a Git/dbt reader for derived-table definitions.